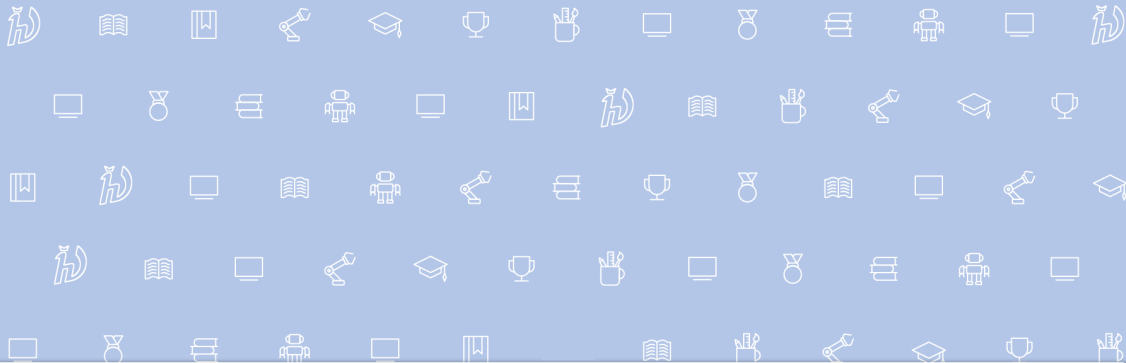




Artificial Intelligence Software, Lecture 8

Experiments & Hyperparameters



Center for
Cognitive
Modeling



Lecture Plan

1. Motivation: why tracking experiments matters
2. Experiment tracking with **ClearML**
3. Visualizing training with **TensorBoard**
4. What are hyperparameters and search strategies
5. Automated HPO with **Optuna**
6. Demo: YOLO + ClearML + Optuna + TensorBoard

`demo/notebooks/demo_hparams.ipynb`



The Experiment Problem

A typical ML workflow involves running many experiments with different configurations.

Without tracking:

- You cannot reproduce a result from last week.
- You lose best checkpoints and their exact configurations.
- Comparing two runs objectively is impossible.

Solution: experiment tracking systems that log everything automatically — code version, packages, config, metrics, artifacts.



ClearML: Task Initialization

```
from clearml import Task

task = Task.init(
    project_name="YOLO-Roboflow",    # groups tasks on dashboard
    task_name="trial - lr0.001",     # human-readable run name
    task_type=Task.TaskTypes.training, # optional: training/testing/...
    reuse_last_task_id=False         # always create a new task
)
logger = task.get_logger()
```

Auto-captured without any extra code: git diff, Python packages, stdout/stderr, hostname, GPU/CPU info, script source.

Rule: call `Task.init()` before any other ML code.



ClearML: Logging Scalars

```
logger.report_scalar(title, series, value, iteration)
```

```
# title    -- chart name on dashboard (groups related series)
```

```
# series   -- line name within that chart
```

```
# iteration -- x-axis value (step / epoch)
```

```
logger.report_scalar("Loss", "train", value=0.42, iteration=5)
```

```
logger.report_scalar("Loss", "val", value=0.55, iteration=5)
```

```
logger.report_scalar("mAP50", "val", value=0.71, iteration=5)
```

Result: a “Loss” chart with two lines (*train* and *val*) and a separate “mAP50” chart — all on the same dashboard page.



ClearML: Logging Images & Plots

```
logger.report_image(  
    title="Predictions", series="batch_0",  
    iteration=epoch, image=pred_img)  
# matplotlib figure  
import matplotlib.pyplot as plt  
fig, ax = plt.subplots()  
ax.plot(losses)  
logger.report_matplotlib_figure(  
    title="Loss curve", series="train",  
    iteration=epoch, figure=fig)  
plt.close(fig)
```

Images appear under Task → *Debug Samples*; figures under *Plots*.



ClearML: Hyperparameters & Artifacts

```
# --- Hyperparameters (Task > Configuration) ---
hparams = {"lr0": 1e-3, "batch": 16, "epochs": 30}
task.connect(hparams, name="training_config")
# dict is mutable: ClearML may override values
# from the dashboard before training starts

# --- Upload any file as an artifact ---
task.upload_artifact("confusion_matrix", "cm.png")
task.upload_artifact("val_predictions", predictions_df)

# --- Register a trained model ---
from clearml import OutputModel
output_model = OutputModel(task=task, framework="PyTorch")
output_model.update_weights("best.pt", auto_delete_file=False)
```

Artifacts and models are versioned and downloadable from the dashboard.



ClearML: Tracking Hyperparameters (demo pattern)

```
hparams = {"epochs": 5, "batch": 16, "lr0": 1e-3}
```

```
# Appears under Task > Configuration on the dashboard
```

```
task.connect(hparams, name="training_config")
```

In the demo, this is called per Optuna trial:

```
task.connect(yolo_hparams,  
             name=f"hparams_trial_{trial.number}")
```

Multiple calls with different name create separate config sections on the dashboard.



What is TensorBoard?

TensorBoard is a web-based visualization tool for monitoring training runs.

Key views:

- **Scalars:** loss and metric curves
- **Images:** samples, augmentations, predictions
- **Histograms:** weight and gradient distributions
- **Graph:** model computational graph
- **Projector:** embedding visualization (t-SNE)
- **HParams:** compare HPO trials side-by-side

Launch: `tensorboard --logdir runs/`



TensorBoard: Writing Logs

```
from torch.utils.tensorboard import SummaryWriter
```

```
writer = SummaryWriter("runs/cnn_training")
```

```
writer.add_scalar("Loss/train", loss.item(), epoch)
```

```
writer.add_scalar("Accuracy/val", val_acc, epoch)
```

```
writer.add_graph(model, dummy_input)
```

```
for name, param in model.named_parameters():
```

```
    writer.add_histogram(name, param, epoch)
```

```
writer.close()
```

See [demo/notebooks/tensorboard_showcase/showcase.ipynb](#)
for a working CNN example with all TensorBoard views enabled.



What Are Hyperparameters?

Hyperparameters are *values set before training* that are not learned from data.

Group	Examples
Optimization	Learning rate, weight decay, optimizer type
Batching	Batch size, gradient accumulation steps
Architecture	Dropout rate, number of layers, activation functions
Regularization	Augmentation strength, label smoothing
Schedule	Warmup epochs, total epochs, early-stop patience
Wrong hyperparameters can prevent convergence entirely — even with a correct architecture.	

Hyperparameter Search Strategies

Strategy	Description
Grid Search	Try all combinations. Exhaustive; scales exponentially with dimensions.
Random Search	Sample uniformly at random. Surprisingly competitive for high-dimensional spaces.
Bayesian (TPE)	Build a probabilistic model of $f(\text{hparams})$; suggest promising regions. Most efficient.
Pruning (ASHA)	Start many trials cheaply; kill under-performers early. Saves 3–10× compute.

Practical default: **Bayesian search + ASHA pruning.**



Optuna: Core Concepts

```
import optuna
```

```
def objective(trial):  
    lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)  
    batch = trial.suggest_categorical("batch", [8, 16, 32])  
  
    score = train_and_evaluate(lr, batch) # returns mAP / accuracy  
    return score  
  
study = optuna.create_study(direction="maximize")  
study.optimize(objective, n_trials=20)  
  
print(study.best_params) # {'lr ': 0.0031, 'batch ': 16}
```

Study: optimization run (history of all trials). **Trial:** one objective call with a sampled config.



Optuna: Search Space API

Log-uniform float -- ideal for learning rates

```
lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)
```

Uniform float -- dropout, weight decay, etc.

```
dropout = trial.suggest_float("dropout", 0.0, 0.5)
```

Integer -- number of layers, warmup steps

```
n_layers = trial.suggest_int("n_layers", 1, 5)
```

Categorical -- discrete choices

```
optimizer_name = trial.suggest_categorical(  
    "optimizer", ["adam", "sgd", "adamw"]  
)
```

In the demo: lr is log-uniform, batch $\in \{8, 16\}$ is categorical.



Optuna: Pruning (Early Stopping of Bad Trials)

Pruning monitors intermediate values and terminates weak trials early.

Workflow:

1. Report intermediate value during training:
`trial.report(mAP50, step=epoch)`
2. Check whether to stop:
`if trial.should_prune(): raise optuna.TrialPruned()`
3. Optuna's pruner decides based on all trials seen so far.

ASHA (Asynchronous Successive Halving) — standard pruner: promotes top-k% of trials at each stage; discards the rest.

Result: spend GPU time only on configurations that are promising early on.



Optuna + TensorBoard Integration

```
from optuna.integration.tensorboard import TensorBoardCallback
```

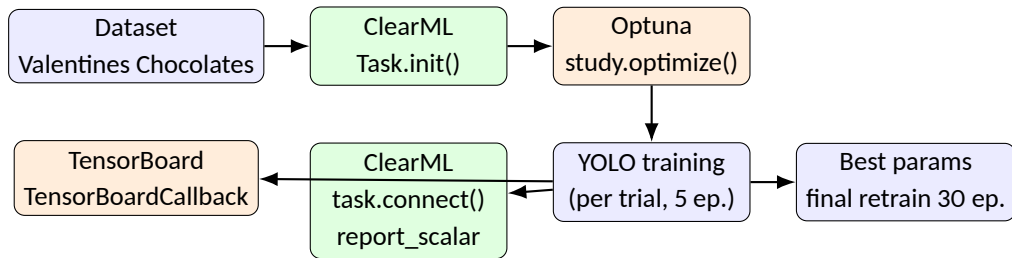
```
tb_callback = TensorBoardCallback(  
    "runs/tb_logs",  
    metric_name="mAP50" )
```

```
study.optimize(  
    objective ,  
    n_trials=3,  
    callbacks=[tb_callback ])
```

Each trial is logged as a separate run under runs/tb_logs/.

Open TensorBoard → *HParams* tab to compare all trials side-by-side.

Demo Pipeline Architecture





Demo Walkthrough: demo_hparams.ipynb

File: demo/notebooks/demo_hparams.ipynb

1. **Init ClearML Task** — `Task.init("YOLO-Roboflow", "Valentines-Chocolates")`
2. **Load dataset** — 22-class Valentines Chocolates in YOLOv8 format
3. **Define Optuna objective** — search lr (10^{-5} - 10^{-2} , log-uniform) and batch ($\in \{8, 16\}$)
4. **Per-trial logging** — `task.connect(hparams) + logger.report_scalar("optuna", "mAP50", ...)`
5. **TensorBoard callback** — writes each trial to `runs/tb_logs/trial-N/`
6. **Final training** — 30 epochs with `study.best_params`
7. **Inference + logging** — predictions annotated and logged to ClearML

Run: `cd demo && pixi run jupyter notebook`



Practical HPO Workflow

1. **Baseline first:** train with default hyperparameters; log everything to ClearML.
2. **Identify key hyperparameters:** 5-10 random trials reveal which params matter most (usually lr and batch).
3. **Narrow and refine:** run Bayesian search (TPE) in the promising region.
4. **Enable pruning:** ASHA eliminates wasted compute in the first epochs of bad trials.
5. **Compare in ClearML:** dashboard side-by-side comparison across all trials.
6. **Final run:** retrain with the best config for the full epoch budget.



Summary

- Experiment tracking is not optional — it is the foundation of reproducible ML.
- **ClearML** auto-captures code, config, metrics, and artifacts for every run.
- **TensorBoard** gives live visual monitoring: loss curves, weight histograms, embeddings.
- **Optuna** turns hyperparameter search into systematic Bayesian optimization.
- Pruning (ASHA) makes HPO practical: spend GPU time only on promising trials.
- The demo (demo_hparams.ipynb) shows the complete pipeline on a real detection dataset.

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521